KernelMind Assignment 4: The Direct Neural Scheduler

Implementation summary

This submission implements a direct-control CPU scheduler. The agent chooses a specific ready-queue process each tick, not a hand-written heuristic. The code includes the extended process model, workload generators, direct MDP environment, DirectSchedulerNet, Double DQN loop, baseline simulators, metrics, plots, and optional-objective evaluators.

- Process includes virtual_runtime, the cumulative CPU ticks actually received by each process.
- The state tensor encodes the top N=10 ready processes with normalized remaining time, wait time, priority, virtual runtime, and arrival time. Unused slots are zero-padded.
- The validity mask is used twice: as an attention key mask and as an output action mask that forces padded Q-values to a very negative value.
- The environment rejects invalid padded actions, executes one tick, updates waits and virtual runtime, admits arrivals, computes reward, and returns next state, mask, reward, and done.
- Training uses replay, target network, epsilon-greedy decay, Huber loss, gradient clipping, and Double DQN target computation.

Reward function

The reward is bounded by construction: tick penalty -0.020, queue penalty in [-0.040, 0], completion bonus +0.800, virtual-runtime variance penalty in [-0.180, 0], and starvation-cap penalty in [-0.250, 0]. The fairness term uses variance of virtual_runtime so it penalizes uneven accumulated CPU service, not just raw waiting.

Answers to required design questions

Design Question 1 - Cost of direct control

The direct action space makes credit assignment harder because the agent chooses among up to N raw processes every tick. A bad choice at tick 1 of a 60-tick episode can affect roughly the next 59 rewards through changed waits, completions, queue composition, and service imbalance. Bellman backups propagate this blame one step at a time. In Assignment 3, one heuristic decision could control several ticks, so there were fewer decision points and a shorter effective credit chain.

Design Question 2 - Reward shaping

Starvation is addressed by two bounded signals: virtual-runtime variance and the wait-time starvation cap. The virtual-runtime term makes it expensive to repeatedly serve jobs that have already received CPU while another ready job remains underserved. This is less susceptible to the raw-wait trap because running a starved long job immediately reduces accumulated-service imbalance, even though the job may remain in the queue for many ticks.

Design Question 3 - Diagnosing instability

The code logs Q-value magnitude and signed TD-error. In this generated run, initial mean absolute Q was 0.123, the final-window mean absolute Q was 1.744, signed TD-error was -0.003, and Huber loss was 0.014. The five-seed stability check produced mean wait std 1.180, fairness std 0.016, and dominated-by-SJF fraction 1.000. If instability were worse, I would first inspect raw reward-term magnitudes under random policy and signed TD-error/target refresh behavior.

Design Question 4 - Output analysis

The Direct-DQN result is dominated by SJF on the core test set. Direct-DQN mean wait=33.060, fairness=0.304; SJF mean wait=23.858, fairness=0.395. If the neural policy only ties or loses to a one-line heuristic, that is not proof of useful learning; next steps are longer stable training, smaller learning-rate sweeps, prioritized replay, and reward-weight sweeps.

Core benchmark results

The generated run trains for 300 episodes and evaluates all policies on the same fixed universal test set.

Policy	Mean Wait	P90 Wait	Jain Fairness
FCFS	34.960	63.518	0.210
SJF	23.858	58.460	0.395
RR	48.452	71.262	0.454
Random	47.203	73.053	0.406
Direct-DQN	33.060	64.740	0.304

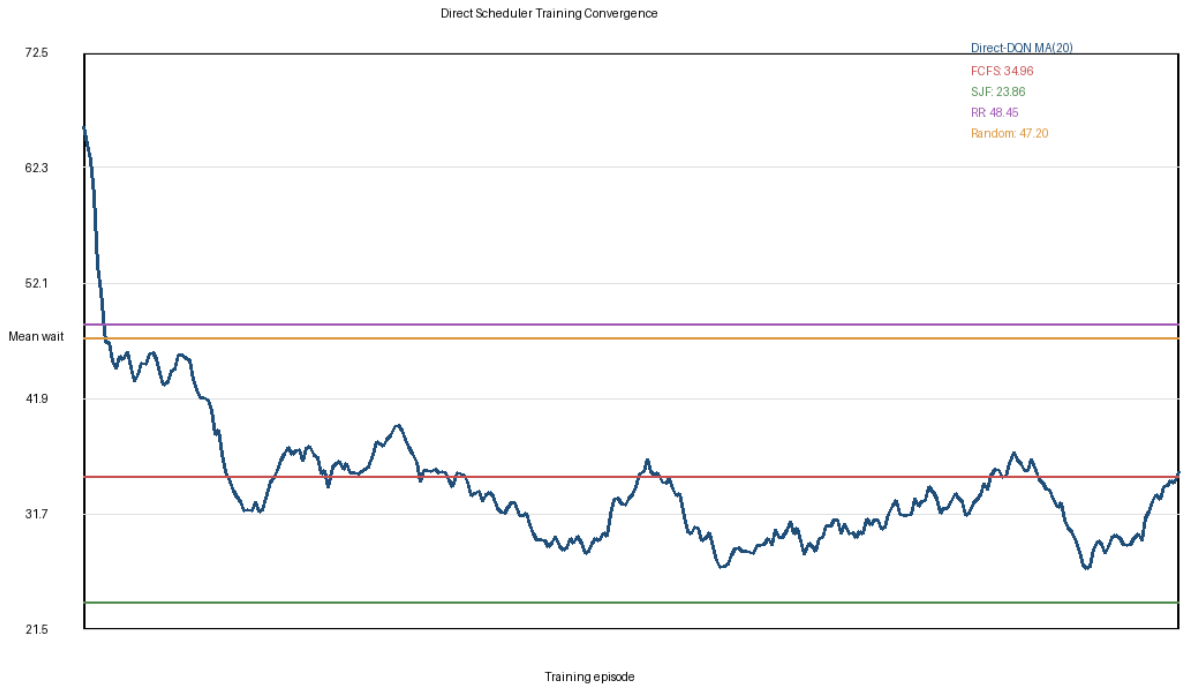

Figure 1. Moving-average training mean wait with baseline reference lines.

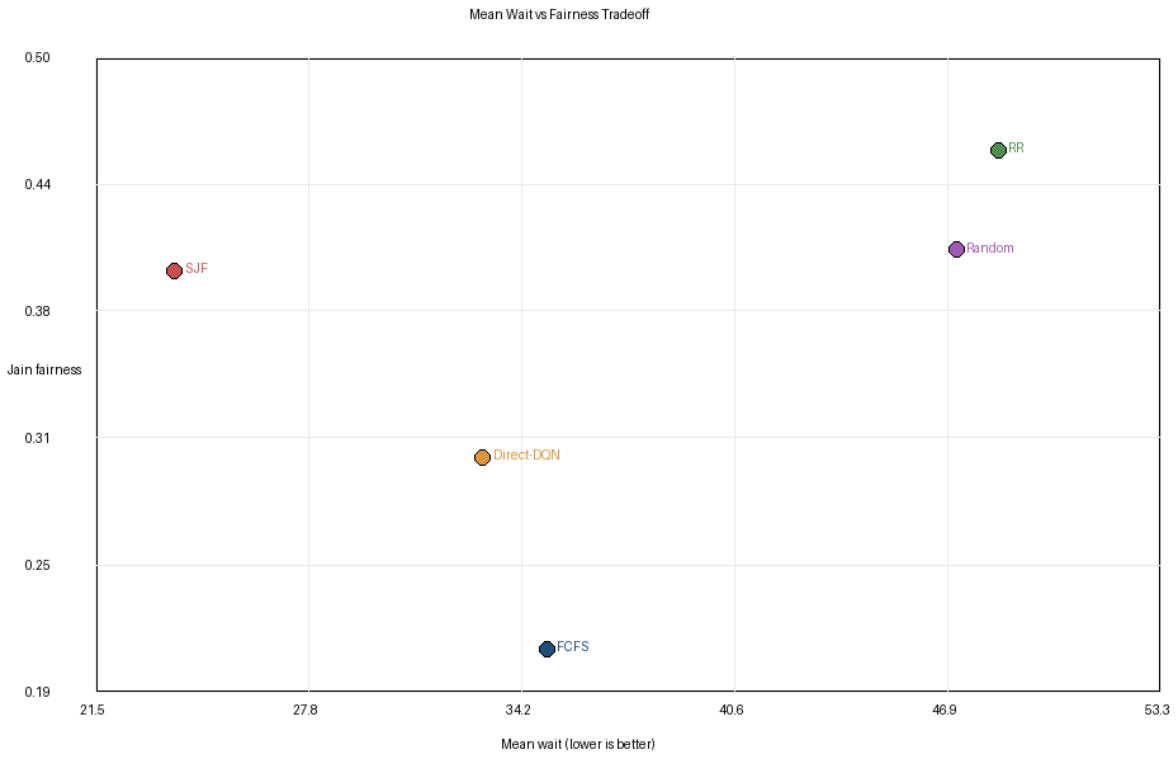

Figure 2. Mean wait versus Jain fairness for all core policies.

Optional objective answers

Symmetric multiprocessing

Implemented a 4-core evaluator. Each policy ranks visible processes once per tick and greedily assigns the top distinct processes to cores. last_core and migrations are tracked, and all policies use the same core-assignment logic.

Policy	Mean Wait	P90 Wait	Jain Fairness	Migrations / Job
FCFS	0.693	1.807	0.905	0.000
SJF	0.550	1.553	0.927	0.433
RR	0.887	1.597	0.889	0.603
Random	0.937	1.983	0.871	0.953
Agent	0.620	1.687	0.913	0.513

Memory/resource constraints

Implemented memory_required, a fixed memory cap, a swap queue, and deterministic first-fit admission every tick. The agent does not receive memory as an input feature, so swap-time differences are indirect consequences of clearing resident jobs.

Policy	Mean Wait	P90 Wait	Jain Fairness	Avg Swap Wait
FCFS	35.443	64.497	0.202	6.733
SJF	24.227	58.027	0.374	3.173
RR	44.600	69.117	0.456	10.970
Random	44.173	69.617	0.437	11.143
Agent	32.850	62.977	0.330	6.077

I/O Storm stress test

The I/O Storm generator creates 8 short jobs with bursts 1-2 and 2 CPU-heavy jobs with bursts 50-60, all arriving simultaneously. The network is evaluated without retraining, so a collapse here indicates out-of-distribution weakness rather than a training-set improvement.

Policy	Mean Wait	P90 Wait	Jain Fairness
FCFS	12.236	17.792	0.424
SJF	11.404	17.656	0.477
RR	17.280	63.956	0.404
Random	19.540	59.600	0.499
Direct-DQN	36.236	58.612	0.274

Seed stability

Seed	Mean Wait	Jain Fairness	Dominated by SJF
11	26.058	0.368	yes
13	27.182	0.353	yes
17	27.798	0.393	yes
19	28.408	0.360	yes
23	25.575	0.359	yes
Mean/Std	27.004/1.180	0.367/0.016	1.000

Submitted files

The archive contains only the requested deliverables: source code, run_all.py, requirements.txt, convergence_plot.png, tradeoff_scatter.png, and report.pdf. It excludes caches, checkpoints, and unrelated intermediate artifacts.